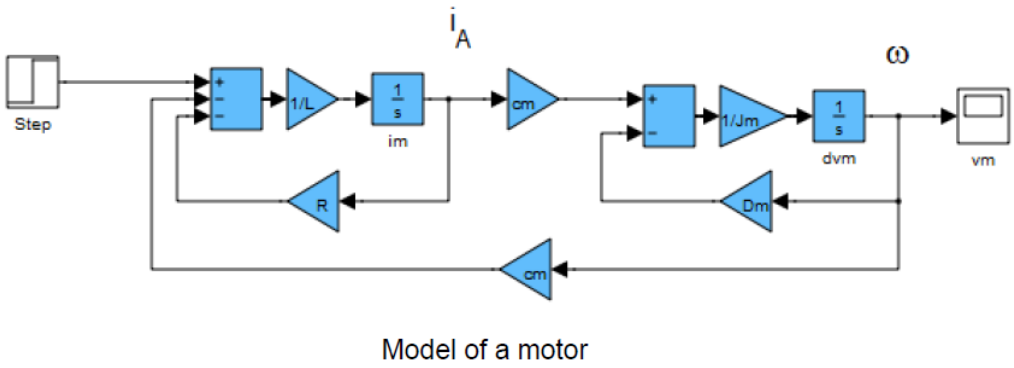
Exercise S-Function
Building Simulink-Blocks in C

Group 10
Rakesh Nuggehalli Ramesh - 15546621
Sakshi Arali - 23046143
Elrick Dsouza - 27846430
Meghana Chitteri - 10546387

In Simulink new blocks can be added using S-functions as so called MEX files. This function can be written e. g. in the C programming language. The advantages of S-Functions are:
*Existing models in C can be translated to Simulink models with little effort
*Model details and know-how can be hidden inside the blocks.
*S-functions are evaluated with better performance
*Control-code written in C can later be used on a microcontroller
In this exercise the model of exercise 4 will be implemented as an S-Function. The gray blocks in the model below are implemented inside an S-Function.



Model of a motor

Figure 1:

a)Implement the model as an S-Function using the S-function builder. Investigate the generated c source code.
b) Test the block and compare the results with the signal-flow model
c) Create your own symbol for the block
d) Investigate further examples of s-blocks inside the S-Function Demo library

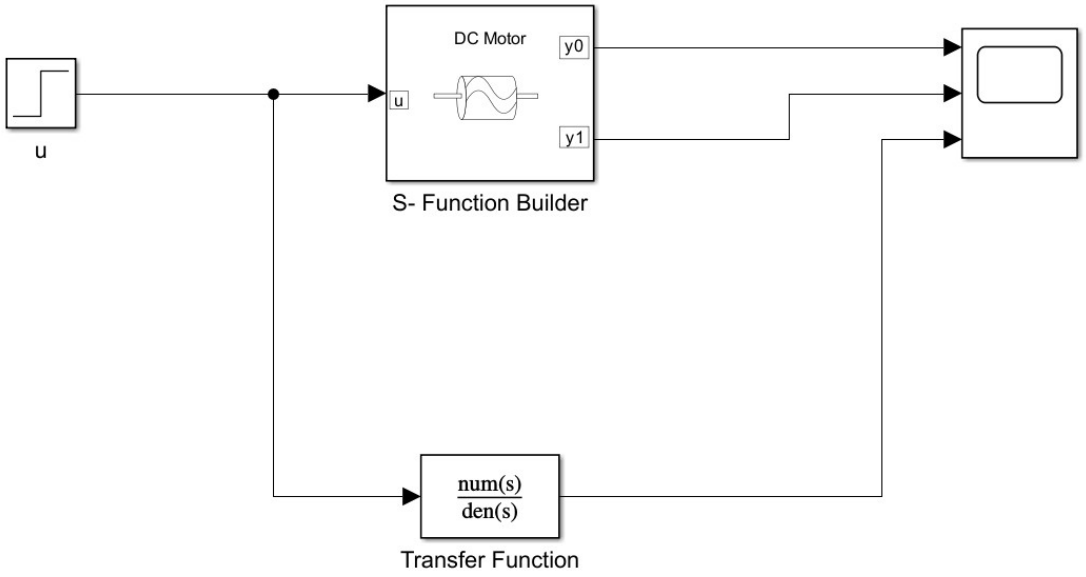


Figure 1: A step input drives a DC motor modeled using an S-Function Builder, and its outputs (current and speed) are compared with a transfer function model using a scope to validate the custom motor model.The S-Function Builder is used to implement the physical (time-domain) model of the DC motor using its differential equations. The input is the applied armature voltage, and the outputs are motor current and speed.

S-FUNCTION CODE

```
    /* Includes_BEGIN */
#include <math.h>
/* Includes_END */

/* Externs_BEGIN */
/* extern double func(double a); */
/* Externs_END */

void Group10_Start_wrapper(real_T *xC,
                           const real_T *L, const int_T p_width0,
```

```
                              const real_T *cm, const int_T p_width1
                                 ,
                              const real_T *R, const int_T p_width2,
                              const real_T *J, const int_T p_width3,
                              const real_T *D, const int_T p_width4)
{
/* Start_BEGIN */
/*
 * Custom Start code goes here.
 */
/* Start_END */
}

void Group10_Outputs_wrapper(const real_T *u0,
                             real_T *y0,
                             real_T *y1,
                             const real_T *xC,
                             const real_T *L, const int_T
                                p_width0,
                             const real_T *cm, const int_T
                                p_width1,
                             const real_T *R, const int_T
                                p_width2,
                             const real_T *J, const int_T
                                p_width3,
                             const real_T *D, const int_T
                                p_width4)
{
/* Output_BEGIN */
y0[0] = xC[0];
y1[0] = xC[1];
/* Output_END */
}

void Group10_Derivatives_wrapper(const real_T *u0,
                                 real_T *y0,
                                 real_T *y1,
                                 real_T *dx,
                                 real_T *xC,
                                 const real_T *L, const int_T
                                    p_width0,
                                 const real_T *cm, const int_T
                                    p_width1,
                                 const real_T *R, const int_T
                                    p_width2,
```

```
                                    const real_T *J, const int_T
                                       p_width3,
                                    const real_T *D, const int_T
                                       p_width4)
{
/* Derivatives_BEGIN */
dx[0] = (1.0 / L[0]) * (u0[0] - R[0]*xC[0] - cm[0]*xC[1]);
dx[1] = (1.0 / J[0]) * (cm[0]*xC[0] - D[0]*xC[1]);
/* Derivatives_END */
}

void Group10_Terminate_wrapper(real_T *xC,
                               const real_T *L, const int_T
                                  p_width0,
                               const real_T *cm, const int_T
                                  p_width1,
                               const real_T *R, const int_T
                                  p_width2,
                               const real_T *J, const int_T
                                  p_width3,
                               const real_T *D, const int_T
                                  p_width4)
{
/* Terminate_BEGIN */
/*
 * Custom Terminate code goes here.
 */
/* Terminate_END */
}
```

MATLAB CODE

```
    %% Excercise S-Function Builder
clear;clc;clf
L = 0.23e-3; %[V/A*sec] Induktivity
cm = 23.4e-3; %[Nm/A] Motor-constant
R = 2.4; %[Ohm] Resistanace
J = 0.23e-6; %[N*m^2] Inertiay
D = 0.4191e-5;%[Nm/rad*sec] Damping

out = sim('Excercise6a');


t = out.Scope.time;
sig = out.Scope.signals;
nSignals = length(sig);

figure
for k = 1:nSignals
    subplot(nSignals,1,k)
    plot(t, sig(k).values, 'LineWidth', 1.2)
    grid on
    ylabel(sig(k).label, 'Interpreter','none')

    if k == 1
        title('1.Current - From S Fucntion ')
    end
    if k == 2
        title('2.Speed - From S Fucntion ')
    end
    if k == 3
        title('3.Signal flow model ')
    end

    if k == nSignals
        xlabel('Time (s)')
    end
end

linkaxes(findall(gcf,'Type','axes'),'x')
```

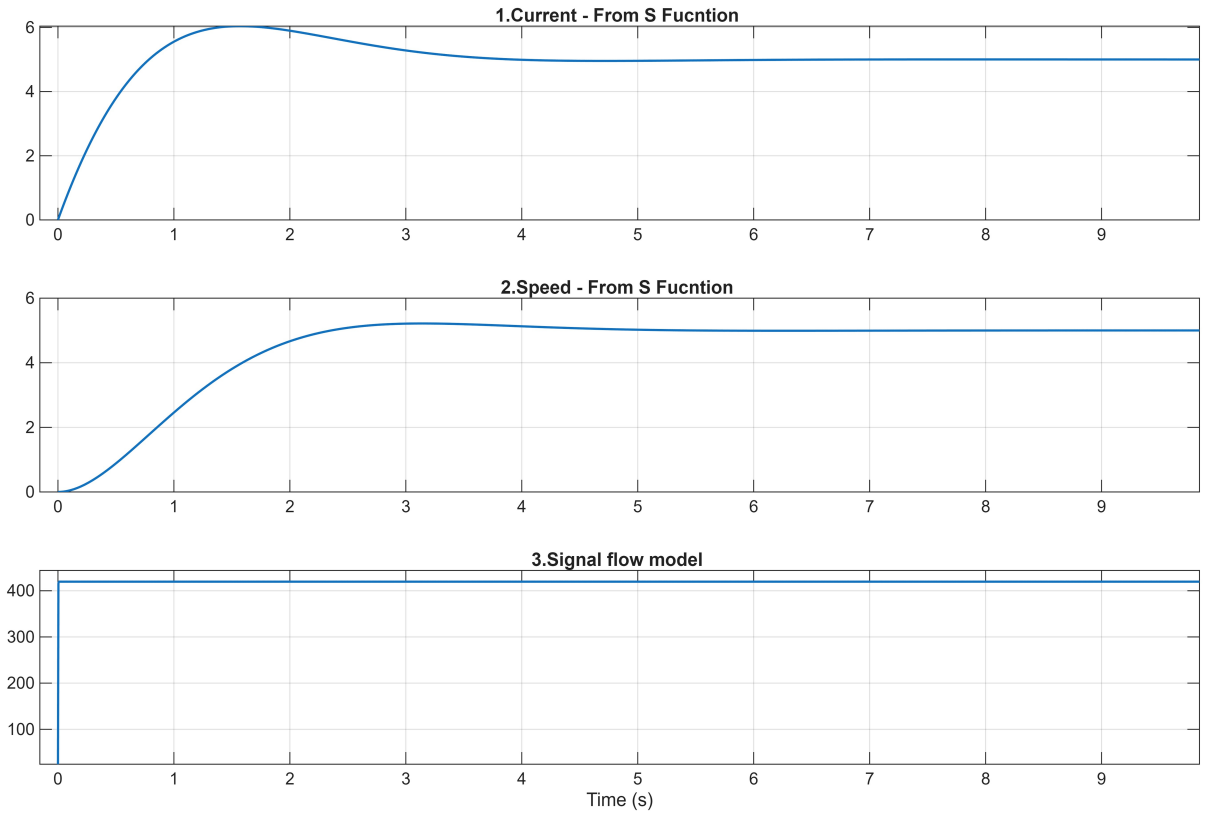


Figure 2: The current and speed of the DC motor rise from zero, show a small overshoot, and then settle to steady values, indicating a stable and well-damped system. The signal flow output remains constant, representing steady-state behavior.